



Circles Social Recovery Audit

Gnosis Ltd - Report by Côme du Crest

2026-04-29



Table of contents

- Table of contents
- Circles Social Recovery Audit
 - Scope
 - Context
 - Status
 - Legal Information And Disclaimer
- Issues
 - [Low] Guardians opting out of recovery may break expired recovery
 - [Low] Missing RecoveryThresholdReached even when lowering threshold
 - [Info] Guardians may no longer be trusted at the time of recovery

Circles Social Recovery Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes all contracts within [aboutcircles/circles-social-recovery](#) as of commit [0xf026b00](#).

The contracts were updated following comments and later reviewed as of commit [0x6e8fc56](#).

Context

The goal of [SocialRecoveryModule](#) is to allow Circles (Gnosis App) users to recover their account by enabling a new passkey as owner on their Safe in case they lost access to their previous passkey.

The social recovery module has to be enabled as a module on the Safe for the recovery to work. Safes then configure “guardians” with a threshold that can come together to enable a new passkey owner.

Passkeys are validated to be proxies deployed by the [SafeWebAuthnSignerFactory](#) from [safe-fndn/safe-modules](#).

Status

The report has been sent to the core developer.

No vulnerabilities have been found and comments have been raised and addressed in messages.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Circles (the “Company”), with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. The auditor has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. The auditor is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, the auditor does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. The auditor disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely my views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. The auditor is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. To the fullest extent permitted by applicable laws, the auditor disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Low] Guardians opting out of recovery may break expired recovery

Summary

`optOutAsGuardian()` allows a guardian to cancel an already expired recovery even when `block.timestamp >= _getRecoveryCooldown(safe) + _getInitiationTimestamp(safe)`. If they are the initiator, the recovery will be removed. Otherwise the count of approvals can be lowered to below threshold.

This is contradictory to `revokeRecoveryApproval()` which is used by guardians to revoke their approval and can only be called before the recovery expires.

Vulnerability Detail

The function `optOutAsGuardian()` removes the recovery if the calling guardian is the initiator and remove the guardian's approval otherwise:

```
1      function optOutAsGuardian(address safe) external onlyConfigured(safe)
2          onlyGuardian(safe, msg.sender) {
3              address guardian = msg.sender;
4              bool activeRecovery = _isRecoveryActive(safe);
5              uint256 threshold = _getThreshold(safe);
6              uint256 guardiansCount = _getGuardiansCount(safe);
7              if (activeRecovery) {
8                  if (guardian == _getInitiator(safe)) {
9                      // if guardian is initiator and opts out - cancel initiated
10                     recovery
11                     _removeRecovery(safe);
12                     emit RecoveryCanceledByInitiatorOptOut(safe);
13                 } else {
14                     mapping(address => address) storage approvingGuardians =
15                     _getApprovingGuardiansList(safe);
16                     if (_isInLinkedList(guardian, approvingGuardians)) {
17                         // if approved recovery - revoke approval
18                         _removeFromLinkedList(guardian, approvingGuardians);
19                         uint256 approvalCount = _getApprovalCount(safe);
20                         uint256 newApprovalCount = approvalCount - uint256(1);
21                         if (approvalCount >= threshold && newApprovalCount <
22                             threshold && threshold <= guardiansCount - 1) {
23                             emit RecoveryThresholdLost(safe);
24                         }
25                         _setApprovalCount(safe, newApprovalCount);
26                     }
27                 }
28             }
29         }
```

```
26      }
```

This `activeRecovery` check does not verify whether the recovery is past its expiry time contrary to `revokeRecoveryApproval()`:

```
1      function revokeRecoveryApproval(address safe) external onlyActiveRecovery(
2          safe) onlyGuardian(safe, msg.sender) {
3          address guardian = msg.sender;
4          if (block.timestamp >= _getRecoveryCooldown(safe) +
5              _getInitiationTimestamp(safe)) {
6              revert RecoveryPeriodEnded();
7          }
8      }
9      ...
10 }
```

Impact

Guardians may expect that once they approved a recovery and the recovery expired it cannot be revoked. They could opt out of the scheme and involuntarily cancel an expired recovery.

Code Snippets

<https://github.com/aboutcircles/circles-social-recovery/blob/6e8fc5617a559e75cff8d022e2ad59af14e2cda9/src/SocialRecoveryModule.sol#L430-L449>

Recommendation

If recovery is expired when opting out, do not lower the count of approval or remove the recovery.

Response

Fixed in commit `0x6e8fc56` by following recommendation.

[Low] Missing RecoveryThresholdReached even when lowering threshold

Summary

When a guardian opts out of an ongoing recovery while they did not approve the recovery, they could lower the threshold for the recovery making the remaining number of guardians reach the threshold. This does not emit the `RecoveryThresholdReached` and will not trigger offchain actions relying on said events.

Vulnerability Detail

If the recovery threshold is equal to the guardians count, a guardian opting out will lower the threshold. This could result in a situation where the threshold is equal to the number of approval. The event `RecoveryThresholdReached` is not emitted in that case:

```
1     function optOutAsGuardian(address safe) external onlyConfigured(safe)
2         onlyGuardian(safe, msg.sender) {
3             address guardian = msg.sender;
4             bool activeRecovery = _isRecoveryActive(safe);
5             uint256 threshold = _getThreshold(safe);
6             uint256 guardiansCount = _getGuardiansCount(safe);
7             ...
8             if (threshold > guardiansCount - 1) {
9                 // enforce threshold drop
10                _setThreshold(safe, threshold - 1);
11                emit ThresholdAutoReducedOnGuardianOptOut(safe, guardian, threshold
                    - 1);
12            } // @audit does not emit RecoveryThresholdReached if reaching
                threshold
13
14            _removeFromLinkedList(guardian, _getGuardiansList(safe));
15            _removeFromLinkedList(safe, _getWardsList(guardian));
16            _setGuardiansCount(safe, guardiansCount - 1);
17            emit GuardianOptedOut(safe, guardian);
18        }
```

Impact

Missing event to trigger offchain actions.

Code Snippets

<https://github.com/aboutcircles/circles-social-recovery/blob/6e8fc5617a559e75cff8d022e2ad59af14e2cda9/src/SocialRecoveryModule.sol#L460-L468>

Recommendation

Add the event if necessary.

Response

The event was added in commit [0x6e8fc56](#).

[Info] Guardians may no longer be trusted at the time of recovery

Summary

`_requireMutualTrust()` only checks the mutual trust at the current `block.timestamp` and is only checked in `_addGuardian()`. That means at the time the passkey is recovered, the Safe could no longer be trusting the guardians.

Vulnerability Detail

The function `_requireMutualTrust()` checks that the guardian trusts the safe and that the safe trusts the guardian. It only checks the current trust status and does not take into consideration the expiry of said trust. It is only checked when the guardian is added:

```
1     function _addGuardian(address safe, address guardian, mapping(address =>
      address) storage guardians) internal {
2         if (guardian == safe) revert SelfGuardianNotAllowed();
3         _requireHuman(guardian);
4         _requireMutualTrust(guardian, safe);
5         _insertIntoLinkedList(guardian, guardians);
6         _insertIntoLinkedList(safe, _getWardsList(guardian));
7         emit GuardianAdded(safe, guardian);
8     }
9
10    function _requireMutualTrust(address guardian, address ward) internal view
      {
11        if (!HUB.isTrusted(guardian, ward)) revert OnlyMutualTrust();
12        if (!HUB.isTrusted(ward, guardian)) revert OnlyMutualTrust();
13    }
```

After a guardian has been added, it can partake in social recovery even though the mutual trust on the hub may be expired or revoked.

Impact

No longer trusted guardians can still participate in social recovery.

Code Snippets

<https://github.com/aboutcircles/circles-social-recovery/blob/6e8fc5617a559e75cff8d022e2ad59af14e2cda9/src/SocialRecoveryModule.sol#L862>

Recommendation

When a guardian approves or initiates a recovery, ensure the trust is still mutual.

Response

The issue has been acknowledged and priority has been given to allow recovery at all times after set up rather than prevent recovery due to no longer trusted guardians. The threshold set in the social recovery scheme should be sufficient to prevent unlawful recoveries.